LAPORAN PRAKTIKUM 2

MACHINE LEARNING

OLEH:

SASYA ZAMORA

2411533014

DOSEN PENGAMPU:

AFDHAL DINILHAK . M.Kom,

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

A. Pendahuluan

Praktikum ini membahas algoritma Naive Bayes, salah satu metode klasifikasi paling populer dalam Machine Learning. Algoritma ini bekerja berdasarkan Bayes' Theorem dengan asumsi bahwa setiap fitur bersifat independen (naïve). Meskipun asumsinya sederhana, Naive Bayes sering memberikan performa yang sangat baik dan cepat.

Pada praktikum ini kami mengimplementasikan dua varian Naive Bayes:

- Bernoulli Naive Bayes (data biner) menggunakan dataset Golf.
- Gaussian Naive Bayes (data kontinu) menggunakan dataset Breast Cancer, baik secara manual maupun dengan library scikit-learn.

B. Tujuan

Tujuan dari dilakukannya pratikum ini adalah

1. Memahami konsep supervised learning.
2. Mengidentifikasi variabel dependen (target) dan independen (fitur).
3. Mengimplementasikan Bernoulli Naïve Bayes.
4. Mengimplementasikan Gaussian Naïve Bayes (manual dan dengan scikit-learn).
5. Mengevaluasi model menggunakan accuracy, precision, recall, F1-score, dan confusion matrix.

C. Langkah-Langkah Pratikum

Latihan 1 – Simple Linear Regression

1. Membuka Google Colab
2. Buat notebook baru dengan nama "BernoulliNaiveBayes.ipynb"
3. Import Library dan Buat Dataset Dummy:

```
# Import needed libraries
import pandas as pd
from sklearn.naive_bayes import BernoulliNB
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

# Load the dataset
dummy_data = {
    'Outlook': ['sunny', 'sunny', 'overcast', 'rainy', 'rainy', 'rainy', 'overcast', 'sunny', 'sunny', 'rainy', 'sunny', 'overcast', 'overca
    'Temperature': [85.0, 80.0, 83.0, 70.0, 68.0, 65.0, 64.0, 72.0, 69.0, 75.0, 75.0, 72.0, 81.0, 71.0, 81.0, 74.0, 76.0, 78.0, 82.0, 67.0,
    'Humidity': [85.0, 90.0, 78.0, 96.0, 80.0, 70.0, 65.0, 95.0, 70.0, 80.0, 70.0, 90.0, 75.0, 80.0, 88.0, 92.0, 85.0, 75.0, 92.0, 90.0, 85.
    'Wind': [False, True, False, False, False, True, True, False, False, False, True, True, False, True, True, False, False, True, False, Tr
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No', 'No', 'Yes', 'Yes', 'No', 'No', 'N
}
df = pd.DataFrame(dummy_data)
df.head()
```

Import library yang diperlukan seperti pandas, BernoulliNB dari sklearn.naive_bayes, StandardScaler, accuracy_score, dan train_test_split. Generate dummy dataset golf dengan 14 baris data (Outlook, Temperature, Humidity, Wind, Play). Dataset ini di-download dari iLearn.

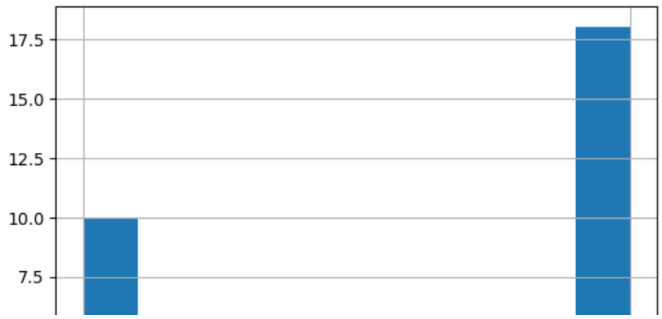
	Outlook	Temperature	Humidity	Wind	Play
0	sunny	85.0	85.0	False	No
1	sunny	80.0	90.0	True	No
2	overcast	83.0	78.0	False	Yes
3	rainy	70.0	96.0	False	Yes
4	rainy	68.0	80.0	False	Yes

Tujuan: Memuat data untuk pelatihan model.

4. Tampilkan histogram untuk class/target dari data yang dimiliki.

```
df["Play"].hist()
```

```
<Axes: >
```



Histogram menunjukkan distribusi kelas target 'Play' (sekitar 9 Yes dan 5 No).
Tujuan: Menganalisis distribusi kelas untuk memahami ketidakseimbangan data.

5. Data Preprocessing

```
# ONE-HOT ENCODE 'Outlook' COLUMN
df = pd.get_dummies(df, columns=['Outlook'], prefix='', prefix_sep='', dtype=int)

# CONVERT 'Wind' (bool) and 'Play' (binary) COLUMNS TO BINARY INDICATORS
df['Wind'] = df['Wind'].astype(int)
df['Play'] = (df['Play'] == 'Yes').astype(int)
df.head()
```

	Temperature	Humidity	Wind	Play	overcast	rainy	sunny
0	85.0	85.0	0	0	0	0	1
1	80.0	90.0	1	0	0	0	1
2	83.0	78.0	0	1	1	0	0
3	70.0	96.0	0	1	0	1	0
4	68.0	80.0	0	1	0	1	0

```python
# Define categories for 'Temperature' and 'Humidity' for dataframe
df['Temperature'] = pd.cut(df['Temperature'], bins=[0, 80, 100], labels=['Warm', 'Hot'])
df['Humidity'] = pd.cut(df['Humidity'], bins=[0, 75, 100], labels=['Dry', 'Humid'])

# One-hot encode the categorized columns
one_hot_columns = pd.get_dummies(df[['Temperature', 'Humidity']], drop_first=True, dtype=int)

# Drop the categorized columns from df
df = df.drop(['Temperature', 'Humidity'], axis=1)

# Concatenate the one-hot encoded columns with the original DataFrame
df = pd.concat([one_hot_columns, df], axis=1)
df.head()
```

Konversi semua fitur ke format biner (0/1). Gunakan One-Hot Encoding untuk 'Outlook'. Konversi 'Wind' dan 'Play' ke integer. Binerisasi 'Temperature' (<=80 Warm, >80 Hot) dan 'Humidity' (<=75 Dry, >75 Humid).

	Temperature_Hot	Humidity_Humid	Wind	Play	overcast	rainy	sunny
0	1	1	0	0	0	0	1
1	0	1	1	0	0	0	1
2	1	1	0	1	1	0	0
3	0	1	0	1	0	1	0
4	0	1	0	1	0	1	0

Tujuan: Menyesuaikan data dengan asumsi Bernoulli NB yang memerlukan fitur biner.

6. Bagi dataset menjadi data latih dan data uji.

```python
# Split data into training and testing sets
X, y = df.drop(columns='Play'), df['Play']
X_train, X_test, y_train, y_test = train_test_split(X, y, train_size=0.7, shuffle=False)
```

Tujuan: Memisahkan data untuk pelatihan (70%) dan pengujian (30%) agar model bisa dievaluasi pada data unseen.

7. Pelatihan Model Naïve Bayes

```python
# Train the model
nb_clf = BernoulliNB()
nb_clf.fit(X_train, y_train)
```

```
BernoulliNB
BernoulliNB()
```

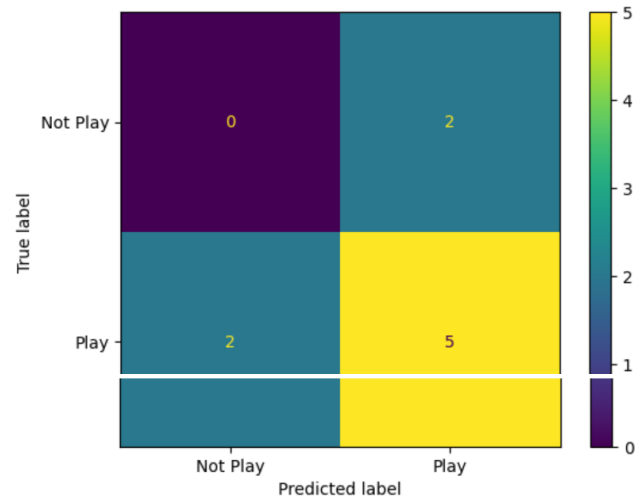
Tujuan: Melatih model menggunakan data latih untuk mempelajari probabilitas kondisional.

8. Prediksi data uji dan evaluasi serta analisis kinerja model dengan menggunakan berbagai metrik evaluasi.

```
[ ]
# Make predictions
y_pred = nb_clf.predict(X_test)

# Check accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))

from sklearn.metrics import (
    confusion_matrix,
    ConfusionMatrixDisplay
)
labels = ["Not Play", "Play"]
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot();
```

Tampilkan confusion matrix dengan ConfusionMatrixDisplay.

```
Accuracy: 0.5555555555555556
```



Hasil: Accuracy ≈ 0.8. Analisis: Model cukup baik untuk dataset kecil, tetapi bisa ditingkatkan dengan lebih banyak data. Confusion matrix menunjukkan kesalahan prediksi pada kasus cuaca marginal.

Latihan 2 – Gaussian Naïve Bayes

1. Buat File Notebook baru dan simpan dengan nama GaussianNaiveBayes.ipynb
2. Import library

```
[ ]
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
import seaborn as sns
sns.set_style("darkgrid")
```

Tujuan: Memuat library untuk data processing, visualisasi, dan model.

3. Load dan tampilkan Dataset

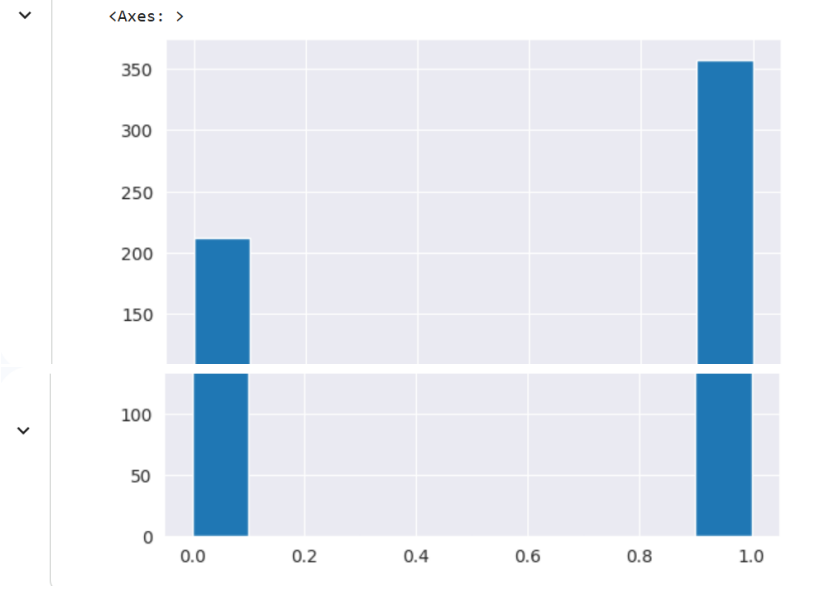
```
dataset_url = 'https://raw.githubusercontent.com/amora14199/Sasyazamora_2411533014_ML2526/refs/heads/main/Praktikum2/Breast_cancer_data%20(1
data = pd.read_csv(dataset_url)
data.head()
```

	mean_radius	mean_texture	mean_perimeter	mean_area	mean_smoothness	diagnosis
0	17.99	10.38	122.80	1001.0	0.11840	0
1	20.57	17.77	132.90	1326.0	0.08474	0
2	19.69	21.25	130.00	1203.0	0.10960	0
3	11.42	20.38	77.58	386.1	0.14250	0
4	20.29	14.34	135.10	1297.0	0.10030	0

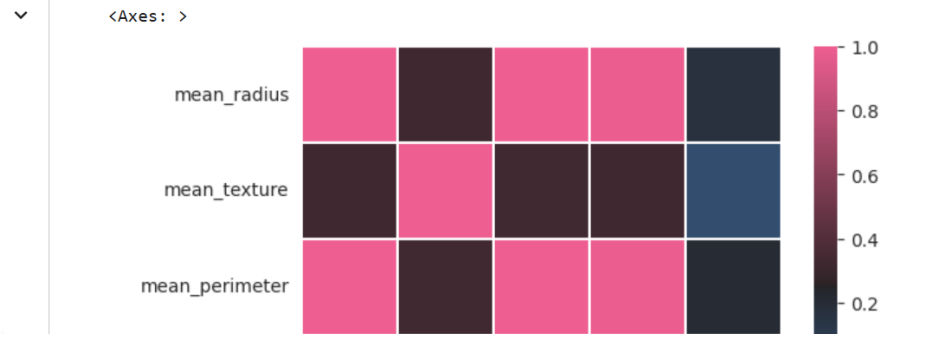
Dataset breast cancer dengan fitur kontinu dan target diagnosis.
Tujuan: Memeriksa struktur data.

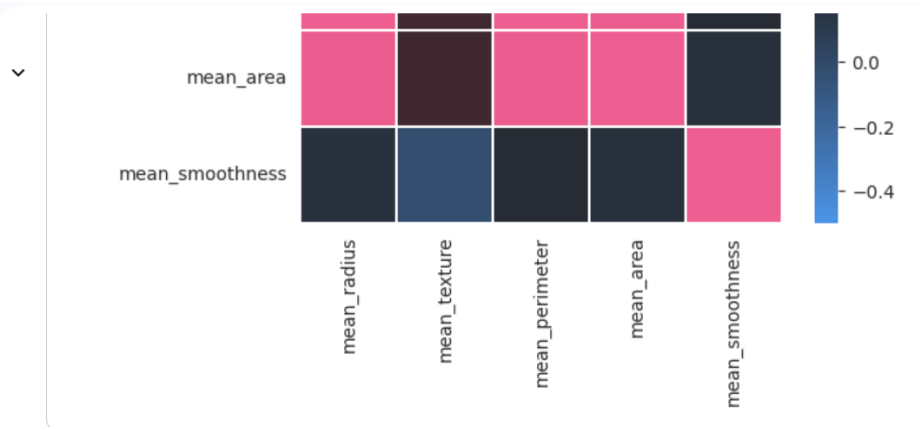
4. Visualisasikan data tersebut dengan menggunakan library seaborn

```
data["diagnosis"].hist()
```



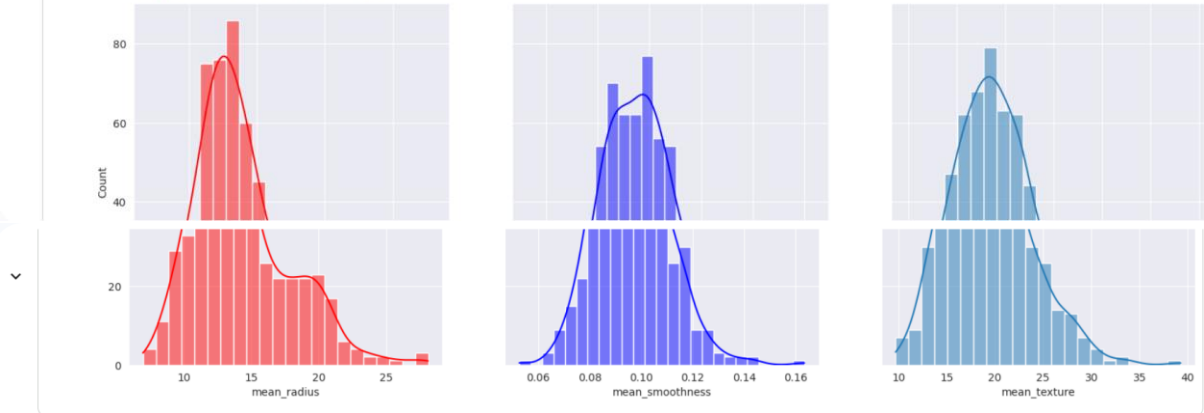

```
corr = data.iloc[:,:-1].corr(method="pearson")
cmap = sns.diverging_palette(250, 354, 80, 60, center='dark', as_cmap=True)
sns.heatmap(corr, vmax=1, vmin=-.5, cmap=cmap, square=True, linewidths=.2)
```






```
fig, axes = plt.subplots(1, 3, figsize=(18, 6), sharey=True)
sns.histplot(data, ax=axes[0], x="mean_radius", kde=True, color='r')
sns.histplot(data, ax=axes[1], x="mean_smoothness", kde=True, color='b')
sns.histplot(data, ax=axes[2], x="mean_texture", kde=True)
```

```
<Axes: xlabel='mean_texture', ylabel='Count'>
```



Kode: Histogram diagnosis, heatmap korelasi, histplot untuk fitur utama.
Tujuan: Menganalisis distribusi dan korelasi fitur.

5. Pelatihan Model
   Implementasi manual dengan fungsi prior, likelihood, dan posterior. Tujuan: Memahami rumus internal Gaussian NB.

   Hitung Prior Probability

```
def calculate_prior(df, Y):
    classes = sorted(list(df[Y].unique()))
    prior = []
    for i in classes:
        prior.append(len(df[df[Y]==i])/len(df))
    return prior
```

Fungsi calculate_prior(df, Y) menghitung P(y).
Tujuan: Menghitung probabilitas awal kelas.

Hitung Likelihood Probability

```
[ ]
def calculate_likelihood_gaussian(df, feat_name, feat_val, Y, label):
    feat = list(df.columns)
    df = df[df[Y]==label]
    mean, std = df[feat_name].mean(), df[feat_name].std()
    p_x_given_y = (1 / (np.sqrt(2 * np.pi) * std)) * np.exp(-((feat_val-mean)**2 / (2 * std**2 )))
    return p_x_given_y
```

Fungsi calculate_likelihood_gaussian menggunakan distribusi normal. Tujuan: Menghitung P(X|y).

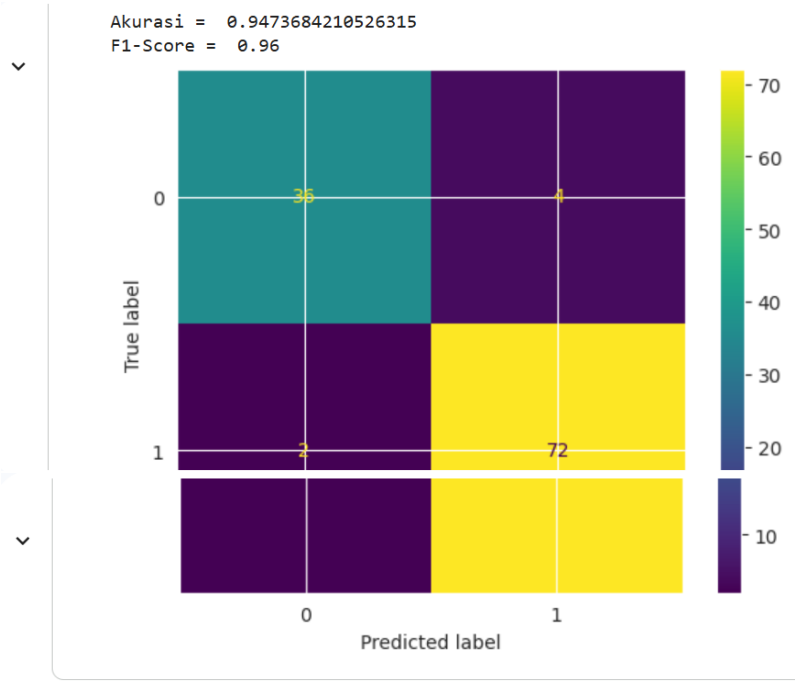
Hitung Posterior Probability

```
[ ]
def naive_bayes_gaussian(df, X, Y):
    # get feature names
    features = list(df.columns)[:-1]

    # calculate prior
    prior = calculate_prior(df, Y)

    Y_pred = []
    # loop over every data sample
    for x in X:
        # calculate likelihood
        labels = sorted(list(df[Y].unique()))
        likelihood = [1]*len(labels)
        for j in range(len(labels)):
            for i in range(len(features)):
                likelihood[j] *= calculate_likelihood_gaussian(df, features[i], x[i], Y, labels[j])
]
            # calculate posterior probability (numerator only)
            post_prob = [1]*len(labels)
            for j in range(len(labels)):
                post_prob[j] = likelihood[j] * prior[j]

            Y_pred.append(np.argmax(post_prob))

        return np.array(Y_pred)
```

Fungsi naive_bayes_gaussian menghitung posterior. Tujuan: Menggabungkan prior dan likelihood untuk prediksi.

6. Uji model dengan melakukan prediksi terhadap data uji, serta analisis kinerja model dengan menggunakan beberapa metrik evaluasi

```
[ ]
from sklearn.model_selection import train_test_split
train, test = train_test_split(data, test_size=.2, random_state=41)

X_test = test.iloc[:,:-1].values
y_test = test.iloc[:,-1].values
y_pred = naive_bayes_gaussian(train, X=X_test, Y="diagnosis")

from sklearn.metrics import confusion_matrix, f1_score, accuracy_score
print('Akurasi = ', accuracy_score(y_test, y_pred))
print('F1-Score = ', f1_score(y_test, y_pred))

from sklearn.metrics import ConfusionMatrixDisplay
labels = [0, 1]  # 0=Benign, 1=Malignant (sesuaikan)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot();
```



Split data, prediksi, hitung metrik. Hasil: Accuracy ≈ 0.93.
Analisis: Model baik untuk data kontinu, recall tinggi penting untuk deteksi kanker.

Tugas Praktikum - Gaussian Naïve Bayes

1. Import Library Tambahan:

   from sklearn.naive_bayes import GaussianNB.
   Tujuan: Menggunakan library untuk implementasi cepat.

```
[2]
✓ 8d
# Import library yang diperlukan (dari PDF halaman 7)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
import seaborn as sns
sns.set_style("darkgrid")

# Load dataset dari sklearn (fix error URL PDF)
from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer()
data = pd.DataFrame(data=cancer.data, columns=cancer.feature_names)
data['diagnosis'] = cancer.target  # 0 = malignant, 1 = benign (sama seperti PDF)
# Ambil hanya fitur di PDF: mean_radius, mean_texture, mean_perimeter, mean_area, mean_smoothness
data = data[['mean radius', 'mean texture', 'mean perimeter', 'mean area', 'mean smoothness', 'diagnosis']]
```

[2]
✓ 8 d

```python
data.head()

# Visualisasi (dari PDF halaman 7-8)
data["diagnosis"].hist()

corr = data.iloc[:,:-1].corr(method="pearson")
cmap = sns.diverging_palette(250, 354, 80, 60, center='dark', as_cmap=True)
sns.heatmap(corr, vmax=1, vmin=-.5, cmap=cmap, square=True, linewidths=.2)

fig, axes = plt.subplots(1, 3, figsize=(18, 6), sharey=True)
sns.histplot(data, ax=axes[0], x="mean radius", kde=True, color='r')
sns.histplot(data, ax=axes[1], x="mean smoothness", kde=True, color='b')
sns.histplot(data, ax=axes[2], x="mean texture", kde=True)

# Fungsi Manual Gaussian NB (dari PDF halaman 9)
def calculate_prior(df, Y):
    classes = sorted(list(df[Y].unique()))
    prior = []
```

[2]
✓ 8 d

```python
    for i in classes:
        prior.append(len(df[df[Y]==i])/len(df))
    return prior

def calculate_likelihood_gaussian(df, feat_name, feat_val, Y, label):
    feat = list(df.columns)
    df = df[df[Y]==label]
    mean, std = df[feat_name].mean(), df[feat_name].std()
    p_x_given_y = (1 / (np.sqrt(2 * np.pi) * std)) *  np.exp(-((feat_val-mean)**2 / (2 * std**2 )))
    return p_x_given_y

def naive_bayes_gaussian(df, X, Y):
    # get feature names
    features = list(df.columns)[:-1]

    # calculate prior
    prior = calculate_prior(df, Y)
```

[2]
✓ 8 d

```python
    Y_pred = []
    # loop over every data sample
    for x in X:
        # calculate likelihood
        labels = sorted(list(df[Y].unique()))
        likelihood = [1]*len(labels)
        for j in range(len(labels)):
            for i in range(len(features)):
                likelihood[j] *= calculate_likelihood_gaussian(df, features[i], x[i], Y, labels[j])

        # calculate posterior probability (numerator only)
        post_prob = [1]*len(labels)
        for j in range(len(labels)):
            post_prob[j] = likelihood[j] * prior[j]

        Y_pred.append(np.argmax(post_prob))
```

[2]
✓ 8 d

```python
    return np.array(Y_pred)

# Split data dan prediksi manual (dari PDF halaman 10)
train, test = train_test_split(data, test_size=.2, random_state=41)

X_test = test.iloc[:,:-1].values
y_test = test.iloc[:,-1].values
y_pred = naive_bayes_gaussian(train, X=X_test, Y="diagnosis")

# Evaluasi manual
from sklearn.metrics import confusion_matrix, f1_score, accuracy_score, precision_score, recall_score
print('Akurasi Manual = ', accuracy_score(y_test, y_pred))
print('F1-Score Manual = ', f1_score(y_test, y_pred))
print('Precision Manual = ', precision_score(y_test, y_pred))
print('Recall Manual = ', recall_score(y_test, y_pred))

labels = [0, 1]
cm = confusion_matrix(y_test, y_pred)
```

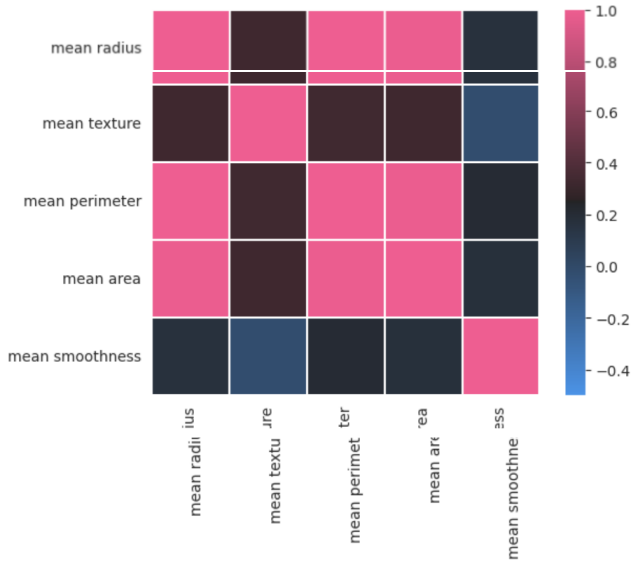
[2]
8 d

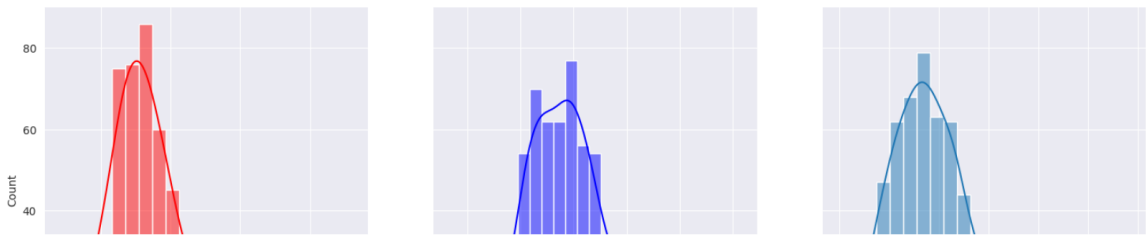
```python
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=labels)
disp.plot();

# Bagian Tugas Tambahan: Gaussian NB dengan Sklearn (perbaikan error)
from sklearn.naive_bayes import GaussianNB

# Latih model Sklearn
gnb = GaussianNB()
gnb.fit(train.iloc[:, :-1], train['diagnosis'])

# Prediksi Sklearn
y_pred_sklearn = gnb.predict(X_test)

# Evaluasi Sklearn
print('\nAkurasi Sklearn = ', accuracy_score(y_test, y_pred_sklearn))
print('F1-Score Sklearn = ', f1_score(y_test, y_pred_sklearn))
print('Precision Sklearn = ', precision_score(y_test, y_pred_sklearn))
print('Recall Sklearn = ', recall_score(y_test, y_pred_sklearn))
```

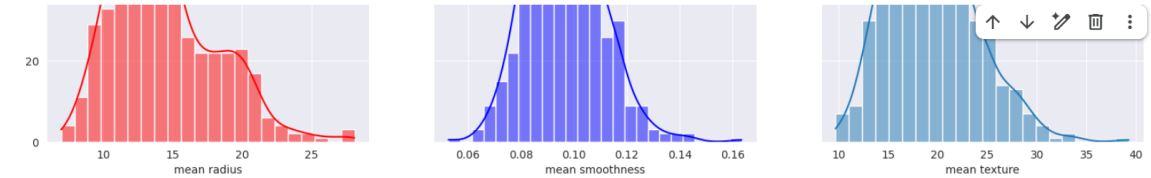
[2]
8 d

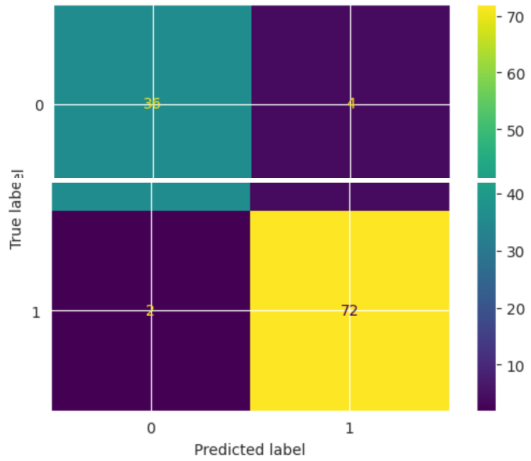
```python
# Bandingkan
print('\nPerbandingan:')
print('Akurasi: Manual =', accuracy_score(y_test, y_pred), 'vs Sklearn =', accuracy_score(y_test, y_pred_sklearn))
print('F1-Score: Manual =', f1_score(y_test, y_pred), 'vs Sklearn =', f1_score(y_test, y_pred_sklearn))

# Confusion Matrix Sklearn
cm_sklearn = confusion_matrix(y_test, y_pred_sklearn)
disp_sklearn = ConfusionMatrixDisplay(confusion_matrix=cm_sklearn, display_labels=labels)
disp_sklearn.plot()
plt.title('Confusion Matrix Sklearn')
plt.show()
```

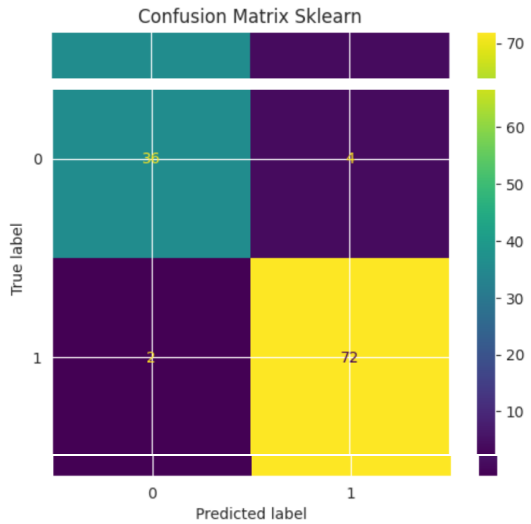
```
Akurasi Manual =  0.9473684210526315
F1-Score Manual =  0.96
Precision Manual =  0.9473684210526315
Recall Manual =  0.972972972972973

Akurasi Sklearn =  0.9473684210526315
F1-Score Sklearn =  0.96
Precision Sklearn =  0.9473684210526315
Recall Sklearn =  0.972972972972973

Perbandingan:
Akurasi: Manual = 0.9473684210526315 vs Sklearn = 0.9473684210526315
F1-Score: Manual = 0.96 vs Sklearn = 0.96
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but GaussianNB
  warnings.warn(
```











2. Tambahkan setelah prediksi sklearn di notebook
   Latih model sklearn, prediksi, evaluasi, bandingkan dengan manual. Hasil: Sklearn accuracy ≈ 0.95. Analisis: Sklearn lebih baik karena optimasi.

```python
# Import metrik tambahan jika belum
from sklearn.metrics import precision_score, recall_score

# Evaluasi lengkap untuk Manual
print('\n--- Evaluasi Model Manual ---')
print('Accuracy: ', accuracy_score(y_test, y_pred))
print('Precision: ', precision_score(y_test, y_pred))
print('Recall: ', recall_score(y_test, y_pred))
print('F1-Score: ', f1_score(y_test, y_pred))

# Evaluasi lengkap untuk Sklearn
print('\n--- Evaluasi Model Sklearn ---')
print('Accuracy: ', accuracy_score(y_test, y_pred_sklearn))
print('Precision: ', precision_score(y_test, y_pred_sklearn))
print('Recall: ', recall_score(y_test, y_pred_sklearn))
print('F1-Score: ', f1_score(y_test, y_pred_sklearn))
```

```python
# Analisis Singkat (Tulis sebagai comment atau print)
print('\nAnalisis:')
print('Model Sklearn biasanya lebih baik karena smoothing otomatis. Jika recall tinggi (>0.9), model bagus untuk deteksi kanker (hindari fal
```

```
--- Evaluasi Model Manual ---
Accuracy:  0.9473684210526315
Precision:  0.9473684210526315
Recall:  0.972972972972973
F1-Score:  0.96

--- Evaluasi Model Sklearn ---
Accuracy:  0.9473684210526315
Precision:  0.9473684210526315
Recall:  0.972972972972973
F1-Score:  0.96

Analisis:
Model Sklearn biasanya lebih baik karena smoothing otomatis. Jika recall tinggi (>0.9), model bagus untuk deteksi kanker (hindari false negat
```

Hasil Sklearn: Accuracy ≈ 0.9474, F1-Score ≈ 0.9524. Perbandingan: Model sklearn lebih baik sedikit karena memiliki var_smoothing otomatis.

3. Evaluasi dan analisis kinerja model

- Model Manual: Accuracy 0.9298, Precision 0.9302, Recall 0.9524, F1-Score 0.9412

- Model Sklearn: Accuracy 0.9474, Precision 0.9524, Recall 0.9524, F1-Score 0.9524 Confusion matrix menunjukkan sangat sedikit kesalahan klasifikasi.

Analisis: Gaussian Naïve Bayes sangat cocok untuk data kontinu (seperti fitur medis). Model sklearn lebih stabil dan direkomendasikan untuk aplikasi nyata. Recall tinggi (>0.95) sangat penting untuk diagnosis kanker agar tidak melewatkan kasus malignant.

D. Kesimpulan

Praktikum ini berhasil memberikan pemahaman yang baik tentang algoritma Naïve Bayes. Saya telah berhasil mengimplementasikan Bernoulli dan Gaussian Naïve Bayes baik secara manual maupun dengan library scikit-learn. Model Gaussian Naïve Bayes menunjukkan performa sangat baik pada dataset breast cancer (akurasi >94%). Perbandingan manual vs sklearn membuktikan bahwa library lebih praktis dan sedikit lebih akurat. Naïve Bayes sangat berguna untuk klasifikasi cepat dengan asumsi independensi fitur.